

The Sixteen Review Questions

Every question to ask of TypeScript code, in order, with where to look, what to plug in, and what the answer means

October 2026

How to use this.

Run the questions in order; it is roughly severity order. Each question is answered by pointing at a line. Each has: where to look, what to plug in, the finding it produces when the answer is wrong, and a one-line way to say it. Questions 1–4 are “does it work at all”; 5–8 “does it work on the edges”; 9–12 “does it survive production”; 13–14 “is it well made”; 15–16 “is my review honest”. Out of time: do 1–8 and say so.

Does it work at all

Question 1. Does it do what the requirement says?

1. Before reading any code, rewrite the requirement as numbered clauses, one testable fact per clause.
2. For each clause, find the line that satisfies it and write the line number next to the clause.
3. A clause with no line is “not implemented” and is always finding number one.
4. A clause whose line does something adjacent but different (lowercases when the spec did not ask; counts all orders when the spec said paid) is “implemented wrong” and ranks with “not implemented”.
5. Named headers, status codes, limits, defaults, ordering and units in the requirement are each a clause.
6. Say it as: “*Clause (d) requires a `Retry-After` header; no line sets one; not implemented.*”

Question 2. Is it a Promise?

1. For every function call, decide: ticket or value.
2. It is a Promise if the function is declared `async`, is annotated `: Promise<...>`, is `fetch` / `res.json` / `db.query` / `new Promise` / `.then` / anything from `fs/promises` or a client SDK, or its body `returns` one of those.
3. If it is a Promise, the call must be `awaited`, `returned` from an `async` function, `.then`’d, or inside `Promise.all` / `allSettled`.
4. A call that is none of those is the missing-await bug; the variable holds the ticket and the next `.property` is `undefined`.
5. `forEach` with an `async` callback never waits; `map` with an `async` callback yields `Promise[]` and needs `Promise.all`.
6. A `try` only catches what is `awaited` inside it; `return f()` escapes, `return await f()` does not.
7. Say it as: “*Line 14 assigns `res.json()` without `await`, so `user` holds a Promise and line 15 caches it.*”

Question 3. Can this be `undefined` here?

1. For every `.something` and every `[index]`, name what is to the left of it and ask whether it can be `undefined` or `null`.
2. Sources of `undefined`: a property marked `?`; `arr.find`; `map.get`; `arr[i]` past the end; `req.headers[...]`, `req.query.x`, `req.params.x`; a function with a path that falls off the end; a destructured key that is absent; a default parameter when the caller passed `null`.
3. Sources of `null`: a database row not found; `JSON.parse("null")`; an explicit `null` from an API.
4. Every `x!` and every `x as T` is the author asserting “cannot be undefined” with no runtime check; verify the claim from the sources above.
5. `?.` makes the access safe but produces `undefined`, which must then be handled by the next line.
6. `if (!x)` treats `0`, `""` and `false` as missing; the precise test is `x == null` or `x === undefined`.
7. Say it as: “*`users.find(...)` can return `undefined`; line 9 reads `.email` on it without a check; crashes for an unknown id.*”

Question 4. Is this `=` really `===`?

1. Inspect every `=` that appears inside `if (...)`, `while (...)`, `? :`, `&&`, `||`, or an arrow passed to `filter` / `find` / `some` / `every`.
2. A single `=` there assigns, returns the assigned value, and is truthy for any non-falsy right-hand side; the branch always runs and the variable is overwritten.
3. Inspect every `==` and `!=`; they coerce (`0 == ""`, `5 == "5"`); expect `===` / `!==`; the one accepted form is `x == null`.
4. Say it as: “*Line 22 `if (o.status = "paid")` assigns; every order is marked paid and the branch always runs.*”

Does it work on the edges

Question 5. What happens at the boundary?

1. For every `<`, `<=`, `>`, `>=`, plug in the limit value itself, then limit minus one, then limit plus one, and say which side each lands on.
2. “At most N” means the check is `>= N` to reject the (N+1)th; `> N` allows N+1.

3. “Fewer than N” means `>= N` rejects the Nth.
4. For every loop, plug in `i = length - 1` and `i = length`; `<= arr.length` runs one past the end and reads `undefined`.
5. For every retry loop, count the attempts: `attempt < 3` starting at 1 is two attempts; `attempt <= 3` or starting at 0 is three.
6. For every `slice(a, b)`, `substring`, `splice(i, n)`: the end is exclusive; `slice(0, n)` gives n items.
7. For every `Math.ceil` / `Math.floor` / `Math.round`: plug in an exact integer and a value just below; `Math.ceil(0.0001)` is 1, `Math.ceil(0)` is 0.
8. For every “rolling window”: a timestamp exactly `window` old is expired or not, and the code must pick one consistently.
9. Say it as: “`recent.length > LIMIT` allows the 101st request; should be `>=`.”

Question 6. What happens on empty, zero, or garbage input?

1. For every parameter and every external value, plug in each of: `undefined`, `null`, `""`, `[]`, `{}`, `0`, `-1`, `NaN`, a huge number, a string where a number was expected, an object with extra keys, an object missing a key.
2. `Number("abc")` is `NaN`; `NaN` fails every `<` / `>` check as `false`, so a range check alone lets it through; require `Number.isInteger` or `Number.isNaN`.
3. `x || default` replaces a legitimate `0`, `""` or `false`; `x ?? default` does not; neither validates.
4. `[] .reduce(f)` with no initial value throws; `Math.max(...[])` is `-Infinity`; `[] .every(...)` is `true`; `[] .some(...)` is `false`.
5. An empty string key, an empty array of ids, a page size of 0, a negative offset, a limit of `1e9`: each is a separate plug-in.
6. A missing required field must be rejected with 400 before use, not defaulted silently.
7. Say it as: “`?size=0` becomes 20 via `||`; `?size=abc` becomes `NaN`, passes `size > 100` as `false`, and reaches the query.”

Question 7. Who else is holding this?

1. For every mutating call (`push`, `pop`, `shift`, `unshift`, `splice`, `sort`, `reverse`, `fill`, `delete obj.k`, `obj.k = v`, `arr[i] = v`, `arr.length = 0`), name the object being mutated.
2. If that object is a parameter, the caller’s data changes.
3. If it is a module-level or class-level variable, every other caller sees the change.
4. If it is a value that was returned to a caller (`return this.list`), the caller can mutate internal state.
5. If it is a cached value, every future cache hit returns the mutated version.
6. `const` does not prevent any of this; it only prevents `x = newThing`.
7. `{ ...o }` and `[...a]` are shallow; nested objects are still shared; `structuredClone` is deep.
8. `filter` copies the array but not the objects inside it; mutating an element of the filtered copy mutates the original element.
9. Say it as: “`orders.sort(...)` on line 30 reorders the caller’s array; the requirement says the input must not be modified.”

Question 8. Did the result get caught?

1. For every non-mutating call (`map`, `filter`, `slice`, `concat`, `flat`, `toSorted`, `toReversed`, every string method, `Object.assign` to a fresh target, `structuredClone`), check that its return value is assigned, returned, or passed somewhere.
2. A non-mutating call alone on a line does nothing.
3. The inverse: `const sorted = arr.sort()` and `const r = arr.reverse()` are not copies; both names point at the same mutated array.
4. `arr.push(x)` returns the new length, not the array; `const a = arr.push(x)` holds a number.
5. `map.set(k, v)` returns the map (chainable); `set.add(x)` returns the set; `delete` returns a boolean.
6. Say it as: “Line 12 calls `name.trim()` and discards the result; the untrimmed name is saved on line 13.”

Does it survive production

Question 9. Where does the error go?

1. For every `throw`, every call that can reject, and every `JSON.parse`, find the nearest enclosing `try`.
2. A `try` catches a rejection only if the call is `await`ed inside the `try` block.
3. A `catch` with an empty body, or one that only logs and continues, swallows the error; a finding unless the requirement says to ignore failures.
4. A `catch (e)` that reads `e.message` without `e instanceof Error` is a type error and a runtime risk.
5. A re-throw that builds a new `Error` without `{ cause: e }` loses the original.
6. `throw "string"` loses the stack trace; expect `throw new Error(...)`.
7. In Express 4, an error thrown inside an `async` handler is not caught by the framework; it needs `next(err)`, a `.catch(next)`, or a wrapper; otherwise the request hangs.
8. `fetch` resolves on 404 and 500; without `if (!res.ok)` the error body is parsed as data.

9. `Promise.all` rejects on the first failure and ignores the rest; `allSettled` reports each.
10. A fire-and-forget async call with no `.catch` becomes an unhandled rejection.
11. Say it as: “`save(data)` on line 8 is inside `try` but not awaited; a rejection escapes the catch and is unhandled.”

Question 10. Is this input trusted?

1. Every value from `req.params`, `req.query`, `req.body`, `req.headers`, a URL, a file, an environment variable, or `JSON.parse` is untrusted.
2. Untrusted value into a SQL string (``... WHERE id = '${id}'``) is injection; expect a parameter (`$1`, `?`).
3. Untrusted value into a URL path without `encodeURIComponent` is a finding.
4. Untrusted value into a shell command, `eval`, `new Function`, or `innerHTML` is a finding.
5. Untrusted value used as a plain-object key (`obj[userKey]`) can hit `__proto__` or `constructor`; expect a `Map`.
6. The authorization check must happen before the work, and must `return` after sending the rejection.
7. The acting user’s id and org must come from the verified token, never from the request body or query; `req.query.orgId ?? claims.orgId` lets the caller choose any org.
8. A secret (`sk_live...`, a password, a token) in source, in a log line, or in an error response is a finding.
9. A stack trace or internal error message returned to the client is a finding.
10. Say it as: “Line 24 interpolates `orgId` into the SQL string; injectable; use `$3`.”

Question 11. What if two of these run at once?

1. Node is single-threaded but every `await` is a point where another request runs.
2. For every shared `Map`, object, array, counter or cache, look for the pattern: read it, then `await` something, then write it.
3. Two concurrent callers both read the old value and both write; one update is lost.
4. Cache check-then-fill (`if (!cache.has(k)) cache.set(k, await load(k))`) lets every concurrent miss call `load`; the fix stores the Promise.
5. A rate limiter or counter that reads, awaits, then increments undercounts under load.
6. A `Map` in module scope is per process; with several instances the limit or cache is per instance; say so once as a design note.
7. Say it as: “Lines 10–12 read the count, await the save, then write `count + 1`; concurrent requests lose increments.”

Question 12. Does this grow forever?

1. For every `Map`, `Set`, array or object that is added to and never removed from, ask who can add and how many distinct keys they can produce.
2. A user-controlled key (API key, user id, query string) with no validation means unbounded growth.
3. A TTL check on read does not evict; entries never read stay forever.
4. A sweep interval only bounds growth between sweeps; a burst of unique keys inside one interval is unbounded.
5. A cap check after insertion allows one over and leaves partial state when it throws; check before inserting.
6. `setInterval` with no `clearInterval` and no `.unref()` keeps the process alive and the closure in memory.
7. Event listeners added per request and never removed accumulate.
8. A per-key array (timestamps, history) with no trimming grows with every request.
9. Say it as: “`hits` is keyed by `x-api-key` with no validation and swept every 10 minutes; a burst of random keys grows it without bound.”

Is it well made

Question 13. What does this return on every path?

1. For every function, write its declared return type (or infer it from `async` and the `returns`).
2. Trace each `return` statement and compare its value with the declared type.
3. Trace the fall-off-the-end path: a loop that may not match, an `if` with no `else`, a `switch` with no `default`; each returns `undefined`.
4. A declared `: User` with a fall-off path lies to callers; a declared `: User | undefined` is honest.
5. Every `=> {` must contain a `return`, or it returns `undefined`; `=> ({ ... })` is the form for returning an object literal.
6. A `void` function whose result is used, and a non-void function whose result is ignored, are both findings.
7. A `forEach` callback’s `return` only skips that one item.
8. A comparator passed to `sort` must return a number; a boolean comparator gives unreliable order.
9. Say it as: “`find` is declared `: User` but returns `undefined` when no id matches; callers crash on `.name`.”

Question 14. Is this the right structure?

1. Only rank this above style when the requirement mentions scale, throughput or latency.
2. `arr.includes(x)` or `arr.find(...)` inside a loop over another array is $O(n \cdot m)$; a `Set` or `Map` makes it $O(n + m)$.
3. Sequential `await`s on independent calls serialize work that could run in `Promise.all`.
4. `shift()` in a loop is $O(n)$ per call; fine at small sizes.
5. `filter` then `map` where one pass was required is minor.
6. A plain object used as a map with user-controlled keys should be a `Map` (see Question 10).
7. A `Record<string, T>` is fine for trusted keys.
8. Say it as: “*B.includes(a)* inside the loop over *A* is $O(n \cdot m)$; a `Set` of *B* makes it linear; minor unless inputs are large.”

Is my review honest

Question 15. What went right?

1. Name at least two real positives before writing the findings.
2. Positives that count: the correct algorithm for the requirement; the right data structure; a guard or validation that exists; a cleanup or eviction job; a parameterized query; clock injection; a discriminated union; a cause-preserving re-throw; clear names.
3. Positives that do not count: “it compiles”; “it is short”; “it has types”; “it is readable”.
4. Say what you checked and found fine, with the value you plugged in: “*Boundary at 100 and 101 behaves per spec.*”

Question 16. Which finding decides the verdict?

1. Rank every finding: (a) violates a requirement clause or crashes on realistic input; (b) wrong on an edge input; (c) security; (d) leak or race; (e) design limitation; (f) efficiency; (g) style.
2. The top finding sets the verdict: (a) → reject or request changes, confidence 1–2; (b)–(d) only → request changes, confidence 2–3; (e)–(g) only → approve with comments, confidence 4–5.
3. Confidence is how sure you are the code meets the requirement, not how sure you are of your findings; never default to 3.
4. Each finding is written as line → consequence → fix, one sentence of consequence.
5. A design limitation is stated once with “acceptable if X; otherwise needs Y”; it is not ranked like a crash.
6. A suspicion you could not confirm is written as “possible, not confirmed: ...” with the reason, and ranked below confirmed findings.
7. Every use of AI during the review is named in one sentence in the Loom.
8. Say it as: “*Confidence 2: the `==` in the status filter breaks clause (a) on every input; request changes.*”

The one-page version

#	Question	Look at	Plug in
1	Does it do what the requirement says?	each clause → a line	the requirement's own numbers and names
2	Is it a Promise?	every call	<code>async</code> , <code>:</code> <code>Promise</code> , <code>fetch/db/.then</code> , callers' <code>await</code>
3	Can this be undefined here?	every <code>.x</code> and <code>[i]</code>	<code>?</code> props, <code>find</code> , <code>map.get</code> , headers, fall-off
4	Is this <code>==</code> really <code>===</code> ?	every <code>==/===</code> in a condition	
5	What happens at the boundary?	every <code><</code> <code><=</code> <code>></code> <code>>=</code> , loop bound, retry count	limit, limit – 1, limit + 1
6	Empty, zero, garbage?	every parameter and input	<code>undefined</code> <code>null</code> <code>""</code> <code>[]</code> <code>0</code> <code>-1</code> <code>NaN</code> <code>"abc"</code> huge
7	Who else is holding this?	every mutation	parameter, shared, returned, cached
8	Did the result get caught?	every non-mutating call	is it assigned? is <code>sort</code> treated as a copy?
9	Where does the error go?	every throw, reject, parse	<code>try</code> with <code>await</code> inside? swallowed? <code>res.ok</code> ?
10	Is this input trusted?	every request value	SQL, URL, shell, HTML, object key, auth order, org from token
11	Two at once?	every shared Map/counter/cache	read → await → write
12	Grows forever?	every add without remove	user keys, TTL on read only, sweep gaps, uncleared interval
13	Returns on every path?	every function	each <code>return</code> , the fall-off, <code>=> {</code>
14	Right structure?	loops and awaits	<code>includes</code> in a loop, sequential awaits
15	What went right?	the whole file	two real positives with a plugged-in value
16	Which finding decides?	your list	rank a–g; top one sets confidence and verdict